EE-170LComputer Programming Lab
Laboratory Session 08
Two-Dimensional Arrays in C++

1 Objective

The students will learn and apply the

- What is a Two-Dimensional Array
- When to use Two-Dimensional Array
- Declaration of Two-Dimensional array
- Initialization of Two-Dimensional Array
- Accessing Array Two-Dimensional element

2 What is an Array?

The simplest form of the multidimensional array is the two-dimensional array. A two-dimensional array is, in essence, a list of one-dimensional arrays. To declare a two-dimensional integer array of size x, y, you would write something as follows

Syntax:

```
type arrayName [ x ][ y ];
```

Where **type** can be any valid C++ data type and **arrayName** will be a valid C++ identifier.

A two-dimensional array can be think as a table, which will have x number of rows and y number of columns. A 2-dimensional array a, which contains three rows and four columns can be shown as below

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Figure 1: Two-dimentional array example

Thus, every element in array a is identified by an element name of the form **a[i][j]**, where **a** is the name of the array, and **i** and **j** are the subscripts that uniquely identify each element in **a**.

3 Intializing Two-dimensional Arrays

Multidimensioned arrays may be initialized by specifying bracketed values for each row. Following is an array with 3 rows and each row have 4 columns.

Example:

```
int a[3][4] =   {
                  {0, 1, 2, 3} ,    /* initializers for row indexed by 0 */
                  {4, 5, 6, 7} ,    /* initializers for row indexed by 1 */
                  {8, 9, 10, 11}    /* initializers for row indexed by 2 */
                }                   ;
```

The nested braces, which indicate the intended row, **are optional**. The following initialization is equivalent to previous example.

Example:

```
int a[3][4] = {0,1,2,3,4,5,6,7,8,9,10,11};
```

4 Accessing Two-Dimensional Array Elements

An element in 2-dimensional array is accessed by using the subscripts, i.e., row index and column index of the array. For example

Example:

int val = a[2][3];

The above statement will take 4th element from the 3rd row of the array.

Example:

```
#include <iostream>
using namespace std;
int main()
{
    // an array with 5 rows and 2 columns.
    int a[5][2] = {{0, 0}, {1, 2}, {2, 4}, {3, 6}, {4, 8}};

    // output each array element's value
    for (int i = 0; i < 5; i++)
        for (int j = 0; j < 2; j++)
        {

            cout << "a[" << i << "][" << j << "]: ";
            cout << a[i][j] << endl;
        }
    return 0;
}
```

LabCodes/lab09p01.cpp

When the above code is compiled and executed, it produces the following result:

Results:

```
a[0][0]: 0
a[0][1]: 0
a[1][0]: 1
a[1][1]: 2
a[2][0]: 2
a[2][1]: 4
a[3][0]: 3
a[3][1]: 6
a[4][0]: 4
a[4][1]: 8
```

Example:

```
#include <iostream>
using namespace std;
int main()
{
    int matrix[5][5];
    for (int m1 = 0; m1 < 5; m1++)
    {
        for (int m2 = 0; m2 < 5; m2++)
        {
            matrix[m1][m2] = 5;
        }
    }
    return 0;
}
```

LabCodes/lab09p02.cpp

Example:

```
#include <iostream>
using namespace std;
int main()
{
    int matrix[2][3];
    // For taking integer inputs in a matrix //
    for (int m1 = 0; m1 < 2; m1++)
    {
        for (int m2 = 0; m2 < 3; m2++)
        {
            cout << "Enter Integer :";
            cin >> matrix[m1][m2];
        }
    }
    cout << endl;

    // For displaying elements of a matrix on a screen //
    for (int m1 = 0; m1 < 2; m1++)
    {
        for (int m2 = 0; m2 < 3; m2++)
{
            cout << "Your Entered Integer are :";
            cout << matrix[m1][m2];
            cout << endl;
        }
    }
    return 0;
}
```

LabCodes/lab09p03.cpp

Lab08: Lab Tasks

1. Two-Dimensional Array Declaration and Initialization
    - Declare a 2D integer array called ”matrix” with dimensions 3x3.
    - Initialize the array with values 1, 2, 3, 4, 5, 6, 7, 8, and 9.
    - Display the values of the array elements using nested loops.
2. Sum of Rows and Columns in a Two-Dimensional Array
    - Declare and initialize a 3x3 integer array with random values.
    - Calculate and display the sum of each row and each column separately.
    - Display the total sum of all the elements in the array.
3. Finding the Maximum Value in a Two-Dimensional Array
    - Declare and initialize a 4x4 integer array with random values
    - Find and display the maximum value in the array.